



Faculty of Engineering, University of Ruhuna

Department of Electrical and Information Engineering

EC 8203: Applied Big Data Engineering — Mini Project Assessment

Statement of Individual Technical Contributions

Project Title: End-to-End Lambda Architecture Data Platform
for Ride-Hailing Fleet Operations

R.M.K.D. Rathnayaka – EG/2021/4758
F.D.S. Warnakulasuriya – EG/2021/4850

1 Executive Summary of Collaboration & Workload Distribution

This project was undertaken as a collaborative two-person engineering effort to architect, implement, evaluate, and document an end-to-end big data platform conforming to the **Lambda Architecture** pattern. The system addresses two core operational challenges in ride-hailing fleet telematics: real-time fleet utilization monitoring and daily batch financial profitability reconciliation.

The workload was divided into a balanced 50% / 50% distribution across the architectural layers:

- **R.M.K.D. Rathnayaka (EG/2021/4758):** Focused on the **Speed Layer & Real-Time Ingestion Pipeline**, encompassing system architecture formulation, continuous telemetry generation via Markov chains, Apache Kafka (KRaft mode) streaming infrastructure, 10-second tumbling window metric aggregations, stateful threshold-based idle alert engine ($t_{\text{idle}} \geq 60\text{ s}$), and real-time integration testing.
- **F.D.S. Warnakulasuriya (EG/2021/4850):** Focused on the **Batch Layer, Serving Infrastructure & Observability**, encompassing the daily batch expense generation model, historical fare and cost reconciliation engine, Apache Airflow DAG scheduling, PyArrow Parquet cold archiving, PostgreSQL serving schema and unified SQL views, FastAPI RESTful serving, Streamlit operational dashboard, and comprehensive technical report compilation.

2 Comprehensive Responsibility Matrix

Table 1 summarizes the breakdown of primary and secondary responsibilities across each tier of the project lifecycle.

Table 1: System Component & Responsibility Matrix

Subsystem / Component	Lead	Support	Key Responsibilities
Architecture & Paradigm Design	Rathnayaka	Warnakulasuriya	Lambda vs. Kappa trade-off analysis, SLA latency budgets, system topology.
Streaming Telemetry Ingestion	Rathnayaka	Warnakulasuriya	Markov state machine, Kafka KRaft broker, topic partitioning by <code>vehicle_id</code> .
Speed Layer Processing	Rathnayaka	—	10s tumbling windows, active/idle count ratios, zone revenues, average speeds.
Real-Time Alert Engine	Rathnayaka	—	Dwell-time state tracking ($t_{\text{idle}} \geq 60\text{s}$), 60s suppression window deduplication.
Batch Expense Simulation	Warnakulasuriya	Rathnayaka	5-minute simulated day clock, fuel burn calculations, random garage service flags.
Batch Reconciliation Engine	Warnakulasuriya	—	Historical JSONL scans, fare-expense joins, net profit, margin %, prescriptive rules.
Airflow Orchestration	Warnakulasuriya	—	DAG dependency graph, periodic task triggers, automated retries, failure alerting.
Columnar Cold Data Lake	Warnakulasuriya	—	PyArrow Parquet schema, dictionary encoding, Snappy compression.
Serving Layer & Schema	Warnakulasuriya	Rathnayaka	PostgreSQL 15 DDL tables, indexes, unified serving SQL view <code>v_unified_vehicle_summary</code> .
API & Interactive Dashboard	Warnakulasuriya	Rathnayaka	FastAPI REST endpoints (<code>/metrics</code> , <code>/alerts</code> , <code>/health</code>), Streamlit live map.
Documentation & Report	Warnakulasuriya	Rathnayaka	LaTeX document structure, TikZ diagrams, Matplotlib result plots, bibliography.

3 Detailed Individual Technical Contributions

3.1 Technical Contributions: R.M.K.D. Rathnayaka (EG/2021/4758)

R.M.K.D. Rathnayaka served as the primary architect and engineer for the streaming ingestion and speed layer subsystems:

1. System Architecture Formulation & Paradigm Evaluation:

- Formulated the theoretical and empirical comparison between Lambda and Kappa architectures, evaluating query latency, recompute cost, state memory footprints, and fault tolerance.
- Defended the adoption of the Lambda Architecture based on the heterogeneous nature of the data sources (continuous streaming GPS vs. periodic daily static garage files) and proved why a pure streaming Kappa engine would suffer state bloat when performing multi-day temporal joins.

2. Streaming Telematics Generator (`data_generator/streaming_producer.py`):

- Engineered a multi-vehicle simulator emulating 25 distinct taxis operating concurrently across four municipal zones (*Downtown*, *Airport*, *Uptown*, and *Suburbs*).
- Implemented a 3-state finite Markov state machine (`idle` → `enroute` → `on_trip` → `idle`) with realistic transition probabilities and zone-specific speed distributions.
- Modeled fare accumulation proportional to elapsed trip duration and destination zone tariffs.
- Embedded resilience logic featuring 15 connection retries with exponential backoff to handle broker restarts.

3. Apache Kafka Infrastructure (KRaft Mode):

- Configured and containerized Apache Kafka in KRaft mode (Kafka Raft Metadata), eliminating the traditional Apache Zookeeper dependency and reducing container startup memory by over 500 MB.
- Designed the topic partitioning strategy using `vehicle_id` as the message key to enforce strict chronological event ordering per vehicle across Kafka partitions.

4. Speed Layer Stream Processing Engine:

- Developed the streaming transformation pipeline utilizing 10-second tumbling windows grouped by geographic zone.
- Implemented real-time aggregations calculating: Active Vehicle Count, Enroute Count, Idle Count, Zone Idle Ratio (%), Aggregate Window Revenue (\$), and Average Fleet Velocity.
- Designed atomic database upsert queries ensuring that streaming metrics continuously update the serving store without concurrency deadlocks.

5. Real-Time Stateful Alert Engine:

- Engineered stateful vehicle dwell-time tracking in memory to identify vehicles remaining in the `idle` state for ≥ 60 seconds.
- Implemented an automated alert insertion mechanism into `speed_vehicle_alerts` paired with a **60-second de-duplication suppression window** to prevent dispatch alert storms.

6. Standalone Execution Engine & Verification (`run_standalone.py`):

- Co-developed the standalone zero-external-dependency execution engine using multi-threaded Python queues and embedded SQLite to enable instant end-to-end evaluation without Docker.

3.2 Technical Contributions: F.D.S. Warnakulasuriya (EG/2021/4850)

F.D.S. Warnakulasuriya served as the primary engineer for the batch layer, cold storage, serving infrastructure, and consumption layer:

1. Daily Batch Expense Generator (`data_generator/batch_generator.py`):

- Designed the simulated clock compression mechanism (1 simulated day = 5 minutes = 300 seconds), enabling complete multi-day operational testing within a standard 15-minute viva demonstration session.
- Formulated the operational expenditure model calculating vehicle fuel consumption based on distance covered (\$0.15 – \$0.22/km) and injected stochastic maintenance events (tire changes, engine tuning, brake repairs) for 20% of the fleet.

2. Batch Layer Profitability Reconciliation Engine (`batch_layer/batch_processor.py`):

- Built the reconciliation engine that scans immutable JSON Lines logs in the Raw Data Lake, aggregating true completed trips and cumulative fare revenue per vehicle.
- Implemented multi-source joins combining historical streaming revenue with daily vendor garage expense extracts on `vehicle_id`.
- Programmed financial accounting algorithms computing Total Expenses, Net Profit (\$), and Profit Margin (%).
- Implemented prescriptive business rules classifying unprofitable vehicles and generating automated operational recommendations ("*GROUND VEHICLE*", "*INSPECT ENGINE*", "*UNDER-UTILIZED*").

3. Apache Airflow DAG Orchestration:

- Designed and configured the periodic DAG (`fleet_daily_profitability_reconciliation`) in Apache Airflow, managing scheduled reconciliation triggers, upstream dependency checks, and failure retries.

4. Cold Data Lake & Parquet Archiving:

- Integrated Apache Arrow / PyArrow to serialize reconciled daily records into Snappy-compressed columnar Parquet files, achieving a 75% disk space reduction compared to raw JSON.

5. Serving Layer Relational Database & Unified SQL Views:

- Architected the PostgreSQL 15 relational schema comprising `speed_fleet_metrics`, `speed_vehicle_alerts`, and `batch_daily_profitability`.
- Designed the unified serving SQL view (`v_unified_vehicle_summary`) that dynamically synthesizes yesterday's reconciled financial ledger with active, unresolved real-time speed alerts.

6. Consumption Layer: FastAPI REST Service & Streamlit Dashboard:

- Implemented the FastAPI service providing asynchronous JSON endpoints for real-time fleet metrics, active alerts, and an automated `/health` probe checking database connection latency.
- Built the interactive Streamlit dashboard featuring live geospatial taxi maps, zone KPI metric dials, and interactive Plotly financial reconciliation charts.

7. Technical Report Compilation & Results Visualization:

- Led the writing and compilation of the comprehensive 15-page LaTeX technical report (`main.tex`), designing the native TikZ architecture diagram and generating high-resolution Matplotlib result charts.